PROGRAMMING IN JAVA

ASSIGNMENT 4 – CO4

TASK 1 – GUI DESIGN REVIEW

Diagnose and Repair a Poor Event-Handling Design

Submission report

Task 1.1 – Diagnosis | Task 1.2 – Redesign | Task 1.3 – Design Defence | Alternative Concept and Complexity Analysis

1. Task 1.1 – Diagnose the Design

The original design uses one ActionListener for all buttons and places validation, database access, waiting and output inside the event handler. The following are its main design deficiencies.

Deficiency	Explanation and user impact
1. Interface and business logic are mixed	The event handler directly validates input, connects to the database and performs payment work. The window cannot be tested independently from the payment/database code, and changes to payment rules require editing GUI code.
2. Incorrect String comparison	The expression e.getActionCommand() == "Pay" compares object references rather than string contents. The Pay branch may not be selected reliably; use "Pay".equals(e.getActionCommand()) or, preferably, distinct listeners/actions.
3. No input validation or exception handling	Empty register numbers, empty amounts, non-numeric values, zero and negative amounts are not handled. NumberFormatException can terminate the handler and the user receives no useful correction message.
4. Database work blocks the event-dispatch thread	The JDBC connection and INSERT execute inside the GUI event handler. While the database responds, the window may stop repainting or responding to clicks.
5. Thread.sleep(3000) freezes the interface	Sleeping on the event-dispatch thread blocks event processing for three seconds. The window appears frozen even if the database operation has completed.
6. No clear user feedback	System.out.println("Paid") writes to the console rather than the window. A user may not know whether the payment succeeded or failed.
7. Receipt button has no implementation	The Receipt button is declared but there is no receipt-specific action. The user cannot view a confirmation or payment details.
8. Unsafe SQL construction	The register number and amount are concatenated into SQL. This can permit SQL injection and can also break when input contains special characters. A PreparedStatement with parameters is safer.
9. Resource and connection lifecycle is unmanaged	The connection is not closed explicitly. Repeated payments can leave database resources open. Use try-with-resources and close resources after each operation.
10. Window closing is not handled	There is no WindowListener to dispose of the frame. The application may remain running after the user tries to close the window.
11. Checked exceptions are not handled	JDBC methods can throw SQLException, and Thread.sleep can throw InterruptedException. The shown handler neither catches nor declares these exceptions, so the code as written will not compile in that form.

2. Task 1.2 – Repair the Design

Design principle: FeeWindow is responsible only for collecting input, responding to button events and displaying status. FeeService owns validation and the payment/database operation. The database operation is called from a worker thread so it does not block the GUI event thread.

2.1 Proposed class structure

Class	Responsibilities
FeeWindow extends Frame	Displays register-number and amount fields, Pay/Clear/Receipt buttons and a status label. Captures events, starts background work and displays the result.
FeeService	Validates register number and amount, opens a JDBC connection, executes a parameterized INSERT and returns a confirmation or throws a meaningful exception.
MySQL fees table	Stores payment records. It is accessed only by FeeService through JDBC.

2.2 Event-handling sequence

1. The user enters a register number and amount, then selects Pay.
2. FeeWindow reads the field values, disables Pay temporarily and displays “Processing payment…”.
3. FeeWindow starts a worker thread and calls FeeService.payFee(regNo, amount) from that thread.
4. FeeService validates the values. If invalid, it throws IllegalArgumentException; otherwise it inserts the payment using PreparedStatement and try-with-resources.
5. The worker posts the result back to the AWT event queue using EventQueue.invokeLater.
6. FeeWindow updates the status label, stores the successful confirmation for Receipt, and re-enables Pay.

2.3 Button responsibilities

Pay: starts validation and payment through FeeService. **Clear:** clears both input fields and updates the status. **Receipt:** displays the last successful payment confirmation; if no payment has succeeded, it displays an explanatory message. **Window close:** a WindowAdapter disposes of the frame.

2.4 Important implementation notes

The SQL uses placeholders (`?`) and setString/setDouble rather than concatenating user input. JDBC resources are closed with try-with-resources. The example uses a MySQL database named `college` and a table named `fees`; configure the URL, username and password in FeeService.java and include MySQL Connector/J when compiling and running. No real bank gateway is included: the example records a payment entry in the local database.

3. Task 1.3 – Design Defence

The redesign is easier to test because FeeService can be tested independently with valid and invalid inputs, without opening a window. FeeWindow only handles presentation and events, so layout changes do not affect payment rules. Payment logic is centralized, reducing duplication and making maintenance safer. PreparedStatement improves database safety, while background execution keeps the interface responsive. Clear class responsibilities also make errors easier to locate and future features, such as a different payment method, easier to add.

Word count: 81 (within the 100-word limit).

4. Alternative Concept and Complexity Analysis

Alternative: give each button its own listener object (or use a Swing Action) instead of one listener with an if–else chain on the action command.

Aspect	One listener + if–else chain	Separate listener/action per button
Event dispatch	For k buttons, the handler may perform up to k command comparisons in the worst case: O(k).	The event source is already associated with its handler; no command-selection chain is needed. Handler selection is treated as O(1) for this design-level comparison.
Listener storage	One listener object: O(1) listener space, plus the conditional code.	k listener objects: O(k) listener space.
Maintainability	A growing chain becomes harder to read and easier to break when commands are added.	Each action is isolated and can be changed or tested separately; more objects are used.

For k buttons, the if–else chain has O(k) worst-case comparison work, while direct association with separate handlers avoids searching through k command strings and is treated as O(1) dispatch selection. The separate design uses O(k) listener objects instead of O(1) listener storage. This is a conceptual comparison: actual GUI event-dispatch overhead depends on the toolkit implementation, and both designs still require O(1) work to invoke the selected handler once it has been identified.

5. Execution and verification checklist

• Create the MySQL database and fees table using create_fees_table.sql.
• Configure DB_URL, DB_USER and DB_PASSWORD in FeeService.java.
• Add MySQL Connector/J to the classpath.
• Compile FeeService.java and FeeWindow.java.
• Run FeeWindow and test valid, empty, non-numeric, zero and negative amounts.
• Test Clear and Receipt, and verify a row is inserted into the fees table after a successful payment.
• Capture screenshots of the window and each relevant result for the assignment repository.

6. Scope and assumptions

The supplied assignment snippet does not define a complete database schema, bank API, receipt format or database credentials. This implementation therefore assumes a MySQL database named `college` with a `fees` table containing `reg_no` and `amount` columns (plus generated payment metadata). It records a fee payment in the database; it does not contact a bank or verify an external transaction. Replace the sample connection settings with the values for your own local MySQL installation.